Task 1 — Web App QA & Debug Report

App under test: RealWorld "Conduit", the public live demo at `https://demo.realworld.show` (Angular frontend, source: `realworld-apps/angular-realworld-example-app`) backed by the public API `https://api.realworld.show/api`.
Tester: Naman Malhotra · **Date:** 10 Nov 2026 · **Browser:** Chrome 156 (desktop 1280×800, phone 375×740)
Method: manual walkthrough of every core flow → automated Playwright run of 38 checks (`task1/qa-tests/qa.mjs`, evidence in `results.json` + `screenshots/`) → each frontend finding confirmed by reading the Angular source.

Summary for stakeholders. The happy paths work: you can sign up, log in, write, edit and delete an article and log out. The app "feels broken" because almost nothing has been built for when things go wrong:

- **When the server is slow or has a hiccup, the app just hangs.** The home page says "Loading articles…" forever. Deleting can fail silently, and a broken link shows a blank white page. Users get no explanation and no retry button.
- **Forms accept bad input and then show confusing messages.** They take a 1-character password and a username made of spaces. A too-long title gives "body title is a string of less than 100 chars". A large post says "check your internet connection" when the internet is fine.
- **Destructive actions aren't protected.** One click deletes an article with no "are you sure?". A double-click sends the request twice.
- **Account security and accessibility need work before production.** The session token is never revoked on logout. Email and password can be changed without the current password. Form fields have no labels for screen readers.

None of these are hard to fix. Most come from one pattern, data requests with no error handling, so a single shared fix addresses several rows at once (see the root-cause analysis).

38	**15**	**5 / 7 / 3**	**9**
automated checks run	issues reported below	High / Medium / Low	checks passed (incl. XSS, auth guard)

Bug table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	**Home feed and tag list hang on "Loading…" forever when the API fails or is slow**	1. Open `/` while the API is down or slow (simulated: `/api/articles` and `/api/tags` return 503). 2. Wait.	**Expected:** after a few seconds, a message such as "Couldn't load articles. Retry". **Actual:** "Loading articles…" and "Loading tags…" stay on screen indefinitely, with no error, no retry and no timeout. Evidence: F1, screenshot F1_feed_stuck_loading.png	High	`article-list.component.ts › runQuery()` subscribes with only a `next` handler, and there is no error state in `LoadingState`. `home.component.ts › tags$` has the same gap. See the root-cause analysis.
2	**Delete article: no confirmation, double-click sends 2 requests, failures are silent**	1. Open one of your articles → click *Delete Article* (or double-click it). 2. Repeat with the DELETE call failing (forced 500).	**Expected:** a confirmation step, a single request, and an error message plus a usable button if it fails. **Actual:** it deletes immediately with no confirmation and no undo. A double-click fired **2 DELETE** requests. On failure, nothing is shown and the button stays greyed out ("disabled" class) but is still clickable. Evidence: D1, D2, D1_delete_failure_silent.png	High	`article.component.ts › deleteArticle()` has no `error` callback and never resets `isDeleting`. The button is "disabled" only through `[ngClass]`, not the `disabled` attribute, and there is no `confirm()`/modal.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
3	**Weak sign-up validation: 1-character passwords and blank or HTML usernames accepted**	Sign up with password `1`; or username `" "` (spaces only); or `<b>qa</b>`.	**Expected:** a minimum password length (for example 8 or more) and a trimmed, safe username. **Actual:** all three are accepted (HTTP 201). Only a 300-character username was rejected (422). The form gives no hint about the rules. Evidence: S2, S3	High	API validation only checks "not empty" and length. The frontend has `Validators.required` only, with no `minLength` / `pattern` and no trimming.
4	**Email and password can be changed without re-entering the current password**	Logged in → *Settings* → change the email (or password) → *Update Settings*.	**Expected:** sensitive changes ask for the current password. **Actual:** `PUT /user` with only the session token returns 200. Anyone with a few seconds at an unlocked laptop, or a stolen token, can take over the account permanently. Evidence: X4	High	The settings form has no "current password" field, and the API's update endpoint doesn't require one. The login token alone authorises everything.
5	**Session token survives logout and is readable by any script**	1. Log in, copy `localStorage.jwtToken`. 2. Click *logout*. 3. Call `GET /api/user` with the copied token.	**Expected:** logout invalidates the session on the server. **Actual:** the UI looks logged out, but the old token still returns 200. The token lives in `localStorage`, so any injected script (XSS, or a compromised third-party script) can steal it. Evidence: O1, O2, X3	High	`user.service.ts › purgeAuth()` only deletes the token locally. There is no logout/revoke endpoint, and `jwt.service.ts` uses `localStorage` instead of an httpOnly cookie.
6	**A missing or deleted article shows a blank white page**	Open `/article/does-not-exist`, or an old link to an article whose title was edited (see #7).	**Expected:** "Article not found" with a link home. **Actual:** only the header and footer render, with no message at all. Evidence: N2, N2_missing_article.png	Medium	A data-shape bug: `article.component.ts` does `errors.set(err.errors)` (the inner object), but `ListErrorsComponent` reads `errors.errors`, so the list is always empty.
7	**Editing an article's title breaks every link shared to it**	1. Publish an article and copy its URL. 2. Edit the title → publish. 3. Open the old URL.	**Expected:** old links keep working or redirect. **Actual:** the slug changes (`qa-article-…` → `qa-article-…-edited`), and the old URL returns 404, then the blank page from #6. Evidence: E1, E2	Medium	The slug is regenerated from the title on every update, and there is no slug history or redirect. Keep the slug stable, or store old slugs and return 301.
8	**Misleading "Unable to connect. Please check your internet connection." for server errors**	Publish an article with a very large body (about 2.4 MB), or hit any API error whose body isn't the expected JSON.	**Expected:** "Your article is too large (max X MB)" or a generic "Server error, try again". **Actual:** "*network Unable to connect. Please check your internet connection.*" while the connection is fine. Evidence: A6, A6_oversized_body.png	Medium	`error.interceptor.ts` treats *any* response without an `errors` JSON object (413, 502 HTML pages, CORS failures) as a network failure. It should branch on `err.status` (0 = offline, 413, 5xx).
9	**Editor has no client-side validation, and error text is cryptic**	1. *New Article* → click *Publish* with everything empty. 2. Enter a 250-character title → *Publish*.	**Expected:** required and max-length rules shown on the fields before submitting. **Actual:** *Publish* is always enabled. Errors only come back from the server, as raw strings like "*body title is a string of less than 100 chars*" (the 100-character limit is shown nowhere). Evidence: A3, A5, A5_long_title_error.png	Medium	`editor.component.ts` creates its `FormControl`s with no validators and no `maxlength`. `ListErrorsComponent` prints `key + message` verbatim.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
10	**Double-clicking "Publish Article" sends two create requests**	*New Article* → fill the form → double-click *Publish Article*.	**Expected:** exactly one request. **Actual:** 2 × `POST /articles` are sent. On a slow connection, impatient clicking causes duplicate posts or a confusing "slug taken" error. Evidence: A7	Medium	`isSubmitting` disables the fieldset only after the next change-detection pass (OnPush and signals), so the second click is already queued. Guard in code with `if (isSubmitting()) return;`.
11	**Signing in on a second device silently logs out the first one**	1. Log in in browser A. 2. Log in as the same user in browser B. 3. Go back to A and open *Settings*.	**Expected:** either multiple sessions are allowed, or A is told "you signed in elsewhere". **Actual:** A's token now returns 401, and the app silently drops A to `/login`, losing any unsaved draft. Evidence: L4, L4_first_session_kicked.png	Medium	The backend issues one active token per user. `error.interceptor.ts` calls `purgeAuth()` on any 401, with no message and no attempt to keep the user's work.
12	**Accessibility: form fields have no labels, the page has no language, and the tag "×" can't be used by keyboard**	Inspect *Sign up*, *New Article* and *Settings* with a screen reader or the Tab key.	**Expected:** every input has a `<label>` or `aria-label`, `<html lang="en">`, and controls can be operated by keyboard. **Actual:** 3/3, 4/4 and 5/5 fields rely on placeholder text only. `lang` is missing. The remove-tag control is an `<i>` with only a click handler (tabIndex −1, no role). Fails WCAG 1.3.1, 2.1.1 and 3.1.1. Evidence: AC-register, AC-editor, AC-settings, AC-tags	Medium	The templates (`auth.component.html`, `editor.component.html`, `settings.component.html`, `index.html`) were written for looks only. Use `<button aria-label="Remove tag">`.
13	**Missing browser security headers on the frontend**	`curl -I https://demo.realworld.show/`	**Expected:** `Content-Security-Policy`, `X-Frame-Options` / `frame-ancestors`, HSTS, `X-Content-Type-Options`, `Referrer-Policy`. **Actual:** none are sent. The app can be framed (clickjacking risk), and there is no CSP to limit damage if an XSS slips through, which matters because the token is in `localStorage` (#5). Evidence: X1	Low	Default nginx config with no security headers. Add them in the server block.
14	**Tags are neither trimmed nor de-duplicated**	*New Article* → enter the tags `qa`, `␣qa`, `qa␣`, `QA` (pressing Enter after each).	**Expected:** one tag, `qa`. **Actual:** 4 separate pills. The saved article has `["qa","QA","qa","testing"]`, which splits the tag sidebar and tag filtering. Evidence: A2, A2_duplicate_tags.png	Low	`editor.component.ts › addTag()` checks `tag.trim() !== ''` but then pushes the *untrimmed* value, and compares case-sensitively.
15	**No "page not found": unknown URLs and other users' edit links render empty screens**	Open `/this-page-does-not-exist`, or `/editor/<someone-else's-slug>`.	**Expected:** a 404 page, or "You can't edit this article". **Actual:** a blank page with only the header and footer, or an empty editor with no explanation. Evidence: N1, E3	Low	`app.routes.ts` has no `**` wildcard route. `editor.component.ts` has no error handler on the article fetch and silently redirects non-authors.

Root-cause analysis — #1: feed stuck on "Loading articles…" forever

What happens. Whenever the articles API is slow, down or returns an error, the home page keeps showing "Loading articles…" (and "Loading tags…") forever. The user sees no error, no retry option and no timeout. To them the site simply looks frozen, which is exactly the "it feels broken" complaint.

Why. In `src/app/features/article/components/article-list.component.ts`, `runQuery()` sets `loading = LOADING` and then subscribes to `articlesService.query()` with only a success callback. When the request fails, the global `errorInterceptor` normalises the error and re-throws it, but nothing catches it in this component. The stream dies, `loading` never leaves `LOADING`, and the template has no branch for a failure because `LoadingState` has no `ERROR` value. The tag sidebar uses the same pattern (`tags$` with only a `tap`), and nothing adds a timeout, so a hung connection also spins forever. It is a classic AI-generated-code gap: the happy path was written and demoed, and the failure path never existed.

```
// today
this.articlesService.query(this.query).subscribe(data => { this.loading.set(LOADED); … });

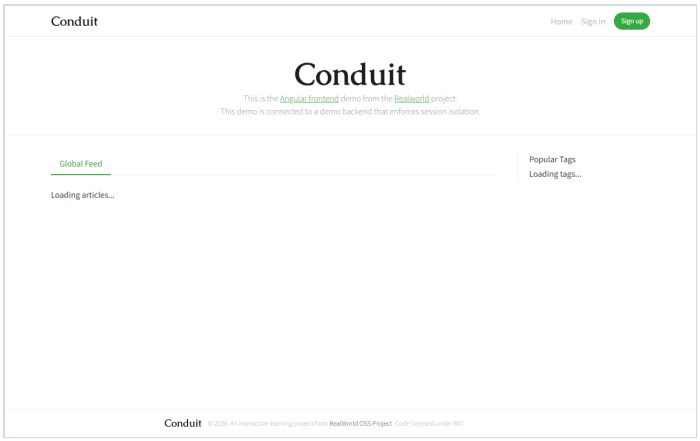
// fix
this.articlesService.query(this.query).pipe(
  timeout(10_000),
  retry({ count: 2, delay: (_, n) => timer(500 * 2 ** n) }),   // GETs are safe to retry
  takeUntilDestroyed(this.destroyRef),
).subscribe({
  next: data => { this.loading.set(LoadingState.LOADED); … },
  error: () => this.loading.set(LoadingState.ERROR),            // template shows "Couldn't load articles · Retry"
});
```

How I'd fix it. Add an `ERROR` state with a “Couldn't load articles · Retry” message, a 10-second timeout, and two automatic retries with backoff (GET requests are safe to retry). Apply the same to `tags$`. Then add an e2e test that forces a 503 with `page.route()`, as check F1 does, so this can't regress. Longer term, a shared “remote data” helper (loading / empty / error / retry) for every list and page would also close #2, #6 and #15. Those bugs share the same root cause: error paths were never designed or tested.

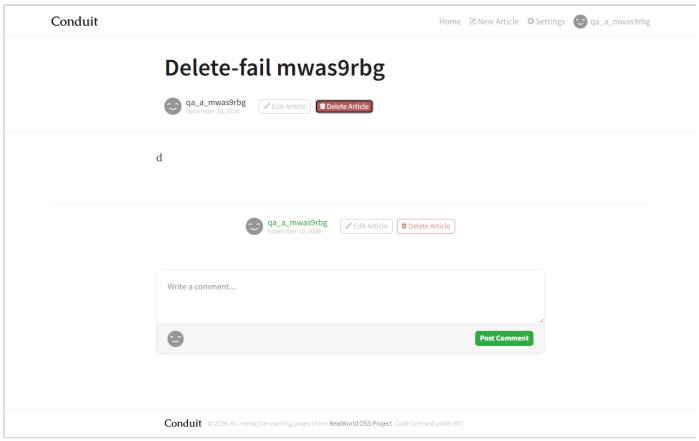
Testing notes & limitations

- **Why the live demo.** The assessment's suggested `demo.realworld.io` returns 404 and the old API returns 530. The project moved to `realworld-apps`, and `demo.realworld.show` is its official replacement. Testing a live deployment also lets the bonus uptime monitor watch a real public URL.
- **Session isolation.** The demo backend deliberately isolates each client's data (“connected to a demo backend that enforces session isolation”). Two observations depend on that sandbox, so I logged them as INFO rather than bugs: (a) signing up with an already-used email is accepted, and the original password stops working; (b) after 12 rapid wrong passwords, even the correct one is rejected with the same “credentials invalid” message (never a 429). Both should be re-tested on a private backend. If they reproduce there, (a) is **Critical** (account takeover) and (b) is a silent lockout.
- **What passed.** Stored-XSS payloads in the title, description and markdown body were rendered harmless by Angular's sanitiser. Protected routes redirect to `/login` after logout. The API blocks editing another user's article. Wrong-password errors are clear. API latency was about 160–235 ms. A throttled 400 kbit/s, 400 ms network still rendered the feed in about 6.6 s.
- **Etiquette.** All test data used throwaway `qa_*@example.com` accounts on the public sandbox, which is meant for exactly this. Created articles were deleted at the end of the run, and the run sent only a handful of requests.

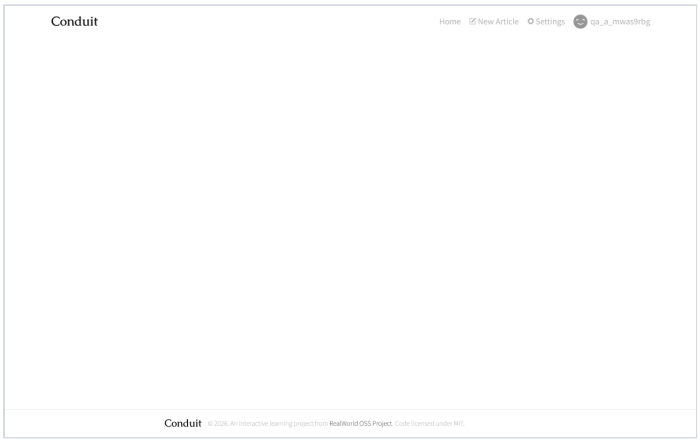
Evidence (selected screenshots)



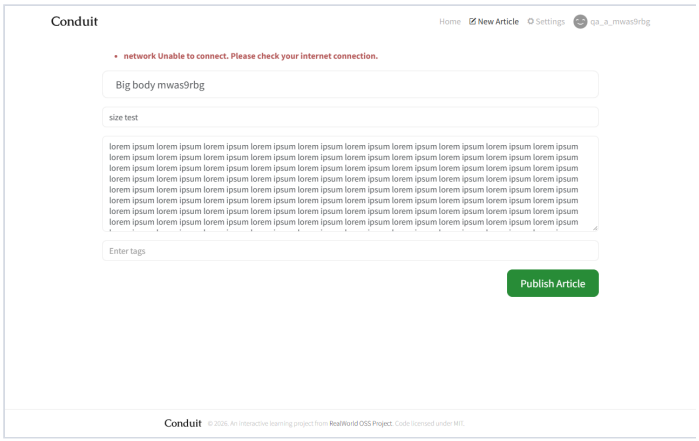
#1 — API returns 503: feed and tags stay on “Loading…” with no error or retry.



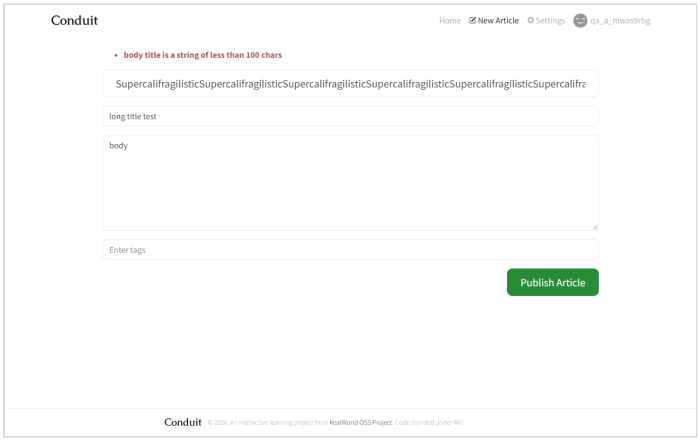
#2 — DELETE failed (500): no message. The user is left on the page with greyed-out but still clickable buttons.



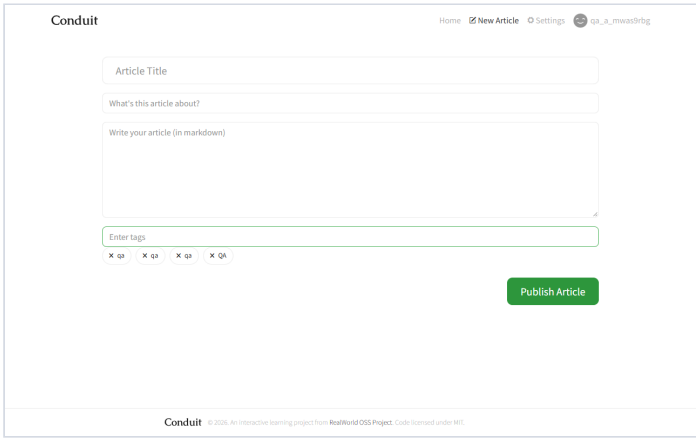
#6 — missing article: blank page, because the error list is always empty.



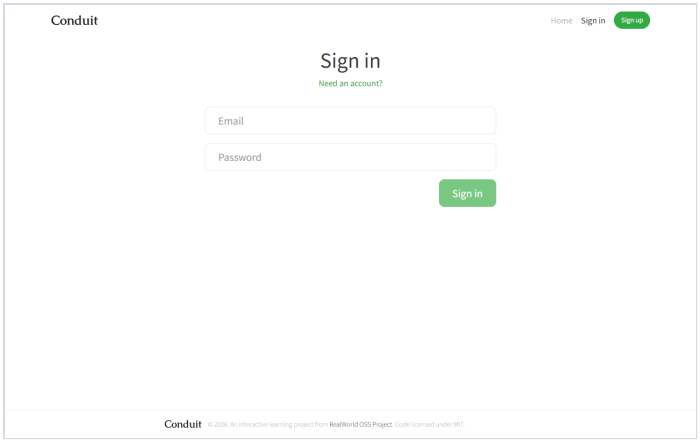
#8 — large post: “check your internet connection” while online.



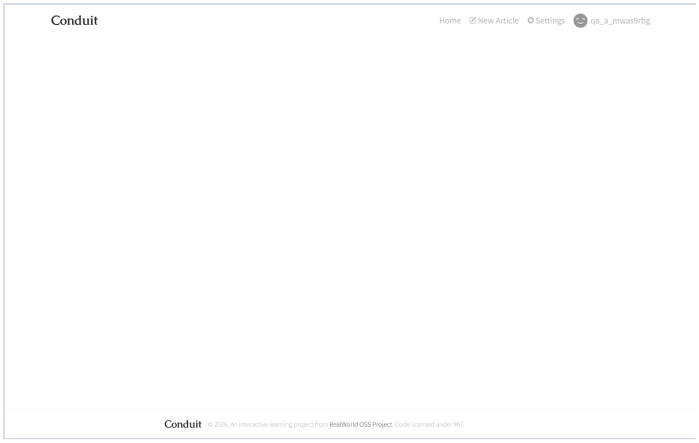
#9 — the title limit is only revealed by a garbled server message.



#14 — “qa”, “ qa”, “qa ” and “QA” become four separate tags.



#11 — the first browser is silently sent to Sign in after a login elsewhere.



#15 — unknown URL: an empty page, with no 404.